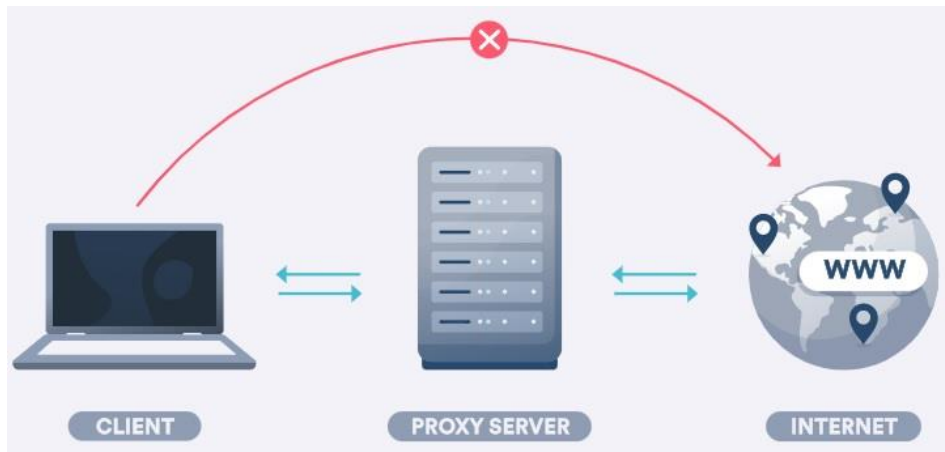


ЗАВДАННЯ

Проксі-сервер. Що це таке, як працює, для чого використовується, та інше:



<https://surfshark.com/uk/blog/proxy-server>

<https://uk.wikipedia.org/wiki/Проксі-сервер>

Використання проксі-серверів зберігає конфіденційність і захищає особисті дані користувачів. Проксі — це проміжний сервер, який виконує функцію посередника між користувачем і іншими віртуальними машинами. Використовуючи даний сервер, ви отримуєте доступ до веб-сайтів більш безпечним шляхом. Окрім того, він може маскувати ваші персональні дані від стороннього спостереження.

Мета завдання.

1. Побудувати модель взаємодії клієнта з проксі-сервером у функціональній частині. Будемо використовувати структуру dict для зберігання *даних показників погоди*, вважаючи що словник (~проксі-сервер) самостійно отримує з інтернету необхідні показники погоди.
2. Реалізувати приклад макета окремих частин проекту, який можна використати як представлення пропозиції (реклама) чи уточнення постановки задачі для розробки. Оцінити окремі кроки розробки проекту.
3. Використати структуру даних dict (словник) для отримання досвіду програмування операцій з словниками практичного наповнення.
4. Навчитись проєктувати словники dict за критеріями швидкодії операцій пошуку даних словника і вибору для запитів. Це важливий крок, зважаючи, що потенційні клієнти можуть звертатись із запитами часто і у великій кількості. Реагування на запит клієнта (швидкість відповіді) є критичною для стабільної роботи будь-якого сервера.

[Для попередньої інформації щодо словника можна переглянути файл "L3-3 Словники-прикладі.pdf" в першій частині.]

Загальні кроки реалізації задачі.

1. Отримати з стороннього джерела дані про погоду (окремо, "вручну"). Покликання на деякі сайти погоди записані в файлі "Сайти даних погоди.txt", цей файл надсилаємо окремо в папку "Матеріали до завдань".
2. Скласти перелік потенційних задач опрацювання даних. На цій основі виконати проєктування словника за критеріями *швидкодії операцій пошуку і вибору даних* із словника. Це важливо, бо визначає ефективність цілого проєкту!
3. Реалізувати кожну задачу програмно як окрему функцію. Можна вважати, що за правильної організації структури словника кожна функція буде складатись з 1-2 рядків програмного коду. Але треба ще врахувати складність обчислювальних операцій, в тому числі прихованих в структурах Python.
4. Визначити спосіб зберігання результатів. Реалізувати збереження і передавання результатів обчислень.
5. Укласти інструкцію користувачу програмного продукту.

Схема реалізації.

1. Для макета реального проєкту замість отримання даних з стороннього джерела можна вводити дані з текстового файлу. Для цього треба, насамперед, точно визначити формат такого файлу. А перед тим, як визначати формат, треба обрати перелік потрібних даних. Наприклад, перелік можна взяти подібно як у зазначеному файлі вище.

1.1. Записати перелік показників погоди, які ви вважаєте за потрібне: країна, місто, час отримання даних, напрям вітру, температура, атмосферний тиск тощо. [Цей перелік в майбутньому може бути переглянутий за побажаннями користувача, тому передбачити можливість неруйнівної модифікації.]

1.2. Точно визначити формат переліку показників, наприклад:

suite ::= info+ NEWLINE

info ::= city delim date delim time delim temperature delim pressure delim
wind NEWLINE

city ::= "Назва міста"

date ::= "month" ":" "date" | "today"

temperature ::= "integer number"

pressure ::= "integer number" | "none"

wind ::= "N" | "NE" | "E" | "SE" | "S" | "SW" | "W" | "NW" | "none"

delim ::= TAB+ | "<->" | "#"

Окремо вирішити питання пробілів в тексті показників.

[*Факультативно.* Задача для активних розробників: що робити, якщо формат вхідних даних не дотримано? Спробувати виправляти чи відхиляти дані?]

1.3. Підготувати один конкретний варіант тестового файлу вхідних даних. Можна "вручну" пошукати метеопказники за сайтами погоди – щоб відібрати тестові дані. [Доведеться трохи попрацювати ☺]. Варто все-таки обирати *реалістичні дані*, а не довільні, щоб можна було візуально оцінити можливості задач опрацювання даних показників погоди.

Для надійного тестування нашої програми, а також для справляння більшого позитивного враження про нашу програму, кількість даних має бути достатньою. Можна для початку записати в тестовий файл вхідних даних 25-30 показників і додавати інші по мірі тестування свого проєкту.

Наголошуємо, що цей файл тестових даних має бути текстового формату, такого, щоб можна було його редагувати для експериментів. В інструкції зазначити джерело отриманих даних (адресу сайту тощо). Програма має *працювати з таким файлом, а не звертатись запитом до сервера*, бо потенційний користувач може мати інші наміри щодо застосування проєкту. Зважити, що ми будемо макет клієнтської частини роботи з проксі-сервером (див. малюнок на початку), отже такий файл є прообразом результату запиту проксі-сервера в інтернет до постачальника даних.

1.4. Визначити структуру словника dict для відображення показників погоди у внутрішню пам'ять. Це *ключовий крок цього завдання*, як зазначено вище в меті завдання – пункт 4. Скласти перелік ключів, полів і значень. Ключі не обов'язково мають дублювати перелік записаних вище показників. Наприклад, за ключем date можна побудувати вкладений словник за ключем time. Таке проєктування полегшить пошук потрібних даних в словнику.

Варто мати на увазі, що вдале проєктування словника автоматично спрощує будову функцій опрацювання словника, і потребує уважного вивчення.

1.5. Скласти функцію читання даних з файлу і збереження даних в словнику. Це імітація отримання запиту від проксі-сервера в інтернет до постачальника даних.

[В майбутньому реальному проєкті таку функцію можна перемкнути від тестового файлу до сервера метеоданих, з яким хоче співпрацювати користувач нашого програмного продукту]

2.1. Приклад переліку потенційних задач опрацювання метеоданих:

- просто відобразити метеодані заданого міста; було б непогано мати можливість вибору формату відображення;
- в якому місті найвища/найнижча температура;
- динаміка зміни температури в зазначеному місті протягом дня;
- який переважаючий напрям вітру в списку визначених міст;
- де температура менша від t – тобто, де настало похолодання.

Очевидно, що подібних задач може бути дуже багато.

Самостійно укласти власний перелік задач з коментарями в програмному коді !

З огляду на потенційні задачі варто повернутись до кроку 1.4 і за потреби внести корекцію структури словника.

2.2. Програмна реалізація задач.

На початку кожну задачу варто реалізувати окремою функцією (в майбутньому вона може стати методом класу – якщо буде сенс будувати клас). Кожна функція може мати необов'язковий параметр (який?) і спосіб повернення результатів.

Результати в загальному випадку можна повертати оператором return (технологія процедурної декомпозиції), або записом в спільну ділянку пам'яті (технологія ООП чи розподілених обчислень). Варто передбачити обидва варіанти.

Переконливо рекомендуємо супроводжувати текст кожної функції розширеними коментарями.

2.3. Виконати внутрішнє незалежне тестування кожної функції. Прийоми тестування функцій розглядали в першому завданні нашого курсу.

3. Спосіб зберігання результатів.

Для макета реального проєкту результати можна записати в текстовий файл. [В майбутньому замість текстового файлу дані можна надсилати користувачам наприклад, за електронною адресою, чи надавати доступ до результатів.]

Отже, треба скласти функцію вибору обчислених даних і перетворення до потрібного формату, який хотів би бачити користувач. [Формат визначаємо подібно до п.1.2]. Реалізувати функцію. Виконати тестування функції.

4. Інструкція користувачу.

Це є звичайний файл текстового формату. Має бути опис призначення програмного продукту, перелік функціональних можливостей, вказівки до виконання функцій програми. Зважити, що інструкцію пишуть для різних категорій користувачів і термінологія інструкції має бути зрозумілою для всіх.

В інструкції не можна вживати терміни з програмування, а тим більше передбачати редагування програми користувачем !

Результати проєкту надіслати в чотирьох файлах:

- інструкція користувача;
- тестовий файл вхідних даних;
- файл програмної реалізації (єдиний!);
- приклад файлу результатів.

Файли надіслати окремими вкладеннями, НЕ АРХІВУВАТИ !